A

Capstone Project Report

On

ZINGLE -Video Calling Realtime Chat App & Social App

Submitted during fifth semester in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Engineering

by

Yash Singh (23001050034)

Under supervision of

Dr. Lalit Mohan Goyal

Department of Computer Engineering
FACULTY OF INFORMATICS & COMPUTING

J.C. Bose University of Science & Technology, YMCA
Faridabad – 121006
July-Dec 2025

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in this Project titled "**ZINGLE -Video Calling Realtime Chat App & Social App**" in fulfilment of the requirements for the degree of Bachelor of Technology in Computer engineering and submitted to "J.C. Bose University of Science and Technology, YMCA, Faridabad", is an authentic record of my own work carried out under the supervision of **Dr**/Ms/Mr **Lalit Mohan Goyal**.

The work contained in this project has not been submitted to any other University or Institute for the award of any degree or diploma by me.

Student's Signature

Yash Singh

CERTIFICATE

It is hereby certified that the project titled **ZINGLE -Video Calling Realtime Chat App & Social App** submitted by Yash Singh (23001050034) of (fifth Semester) to "**J.C. Bose University of Science and Technology, YMCA, Faridabad**" for the award of the degree of Bachelor of Technology in Computer Engineering is a record of a bonafide work carried out by him/her in the Project Lab – J.C. Bose University of Science and Technology, YMCA, Faridabad under my supervision as mentor for Dec-2025 examination.

In my opinion, the project has reached the standards of fulfilling the requirements of the regulations to the degree.

Dr. Lalit Mohan Goyal

Designation

Department of Computer Engg.

ACKNOWLEDGEMENT

I/We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals.

I would like to extend my sincere thanks to all of them. I am highly indebted to my mentor **Dr. Lalit Mohan Goyal** for their guidance and constant supervision as well as for providing necessary information regarding the project & also for their support in completing the project.

I would like to express my gratitude towards my parents & members of J.C. Bose University of Science and Technology for their kind cooperation and encouragement which helped in the completion of this project. I would like to express my special gratitude and thanks to them as well for their guidance.

Student's Signature

Yash Singh

TABLE OF CONTENTS

.

Chapter 1: Introduction

1.1 Introduction of the project

In recent years, digital communication has shifted from traditional messaging to advanced real-time interaction platforms. Today, people prefer instant connectivity where they can chat, talk, and see each other live. To fulfil this modern need, **Zingle** is developed as a Realtime Video Calling and Chat based Social Application.

It provides a platform where users can create their profile, make friends, send messages instantly, and start video calls directly from the chat interface. Zingle ensures smooth, fast and secured communication through modern web technologies like JWT authentication, MongoDB database, React based UI and Stream SDK/WebRTC based calling system.

This project is a step towards building a smart digital social system which enables users to interact just like real life – instantly, visually and anytime from anywhere.

1.2 Problem Statement

Current instant messaging apps are overloaded with unnecessary features and advertisements, which reduces user experience. Also, many simple chat platforms do not provide direct one-to-one video calling along with real-time chats on the

same platform.

Users want a minimal, clean and fast social application which allows:

- easy onboarding
- direct communication
- realtime messaging
- instant video calls

Therefore, there is a need for a lightweight system which combines all these features in a modern UI with good performance and secure backend.

1.3 Objectives of the project

The main objectives of **Zingle** are:

1. To implement a real time chat system using modern web frameworks.
2. To provide video calling functionality directly from the chat screen.
3. To create a social interaction platform where users can add friends.
4. To ensure secure login/registration using JWT authentication.
5. To design an attractive and user-friendly interface using React & Tailwind CSS.
6. To store user data securely in a cloud database (MongoDB Atlas).
7. To deploy the system online so that users can access it from anywhere.

1.4 Learning from the Project

During this project development, several important technical skills and concepts were learned:

- How real-time messaging systems work
- How WebSockets / Stream Chat APIs handle live chat
- How to integrate video calling feature into a web application
- Working with backend APIs (Express.js) and database management (MongoDB)
- JWT based authentication and secure cookie handling
- Frontend component architecture and state management using ReactJS
- Deployment of full-stack apps on cloud platforms

This project helped in improving full-stack development skills and understanding how modern social communication apps are built in real-life industry.

Chapter 2: Literature Review / Related Work

2.1 Related Studies or Existing Systems

In the last decade, several messaging and social networking platforms have been developed that support instant communication. Some of the most relevant systems include WhatsApp, Messenger, Telegram, Discord and Google Meet. These applications provide different features such as messaging, voice/video calling and community-based interactions. However, each platform has certain limitations.

WhatsApp mainly focuses on messaging, but it is tightly coupled with phone number identity and does not provide custom profile-based social interaction or friend discovery features. **Telegram** provides fast messaging but still lacks deeply integrated one-to-one video call management inside chat UI in a developer-friendly manner. **Google Meet** focuses purely on video conferencing and is not designed as a social network or chat-first platform.

Discord is feature rich but is more community/server based and heavy for simple one-to-one personal interactions.

A review of these existing systems shows that most communication platforms are either:

- messaging-first but poor in calling integration, or
- calling-first but poor in social friend-based communication.

Therefore, users looking for a lightweight application that combines messaging + calling + social networking in a minimal UI still do not have a suitable option.

Zingle tries to bridge these gaps.

It combines the advantages of both chat-based platforms and video call platforms into a single minimal, social communication environment. This literature review supports the problem statement that there is still scope for a modern, personal social app which is lightweight, real-time and provides direct calling options in the same channel.

Chapter 3: Methodology

3.1 System/Model Architecture

ZINGLE follows a **Client–Server Architecture** integrated with a third-party real-time communication network.

Key components of Architecture

Component	Role
Frontend (React + Vite)	Handles UI, routing, user interactions
Backend (Node.js + Express)	Handles authentication, REST API, DB queries
MongoDB Atlas	Cloud database for storing users, relationships, requests
Stream API	Enables ultra-fast chat + messaging infrastructure
WebRTC based Video SDK	Used for real-time peer-to-peer audio/video calling

User → Browser (Frontend) → Backend (API) → MongoDB & Stream → Back to Frontend (Real-time UI update)

This architecture ensures:

- Secure user login using JWT
- Real-time chat sync using Stream WebSockets
- Highly scalable communication
- Light and fast application experience

3.2 Dataset/ Data Collection

ZINGLE does not use any external dataset.

It is a **user-driven data system**. All data is generated by the users themselves such as:

- Registration details
- Onboarding profile information
- Friend Requests and acceptance
- Chat messages
- Video call actions/messages

This makes the system dynamic and self-growing.

3.3 Tools and Technologies Used

Category	Tools
Frontend	React.js, Vite, TailwindCSS, DaisyUI, TanStack Query (React Query)
Backend	Node.js, Express.js
Database	MongoDB Atlas
State/Cache Management	TanStack React Query
Authentication	JSON Web Token (JWT), Cookies
Real-Time Chat	Stream Chat API
Video Calling	WebRTC based Stream Video SDK
Deployment Platforms	Render, GitHub

3.4 Algorithms/Techniques Implemented

1. **Friend Match Algorithm**
    - Recommended users are filtered using conditions such as:
        - not same user
        - not already friend
        - user must be onboarded
    - This allows users to only find new people.
2. **Token Generation Technique**
    - JWT is generated at the backend for secure login.
    - Stream-specific token is generated for secure chat connection.

    Real-Time Synchronization

    - Stream WebSockets are used so any message is instantly pushed to both users without page refresh.
3. **ID-based Channel Formation Algorithm**
    - Channel ID for chat is generated using sorted user IDs:

      [myId, targetUserId].sort().join("-")
    - This ensures the same pair always creates the same channel irrespective of who started the chat.
4. **Random Avatar Generator**
    - A random integer (1-100) is selected and used to generate a unique profile picture.

Chapter 4: Implementation

4.1 System Design

Zingle follows a **client–server architecture**:

- **Frontend (Client)** → React-based UI
- **Backend (Server)** → Node + Express API
- **Database** → MongoDB Atlas
- **Third Party Cloud** → Stream API for chat & video

Data flow:

1. User interacts with UI → form/input

   Frontend sends request using Axios/TanStack query
2. Server validates + processes request
3. MongoDB stores data
4. Response is returned to frontend → UI gets updated

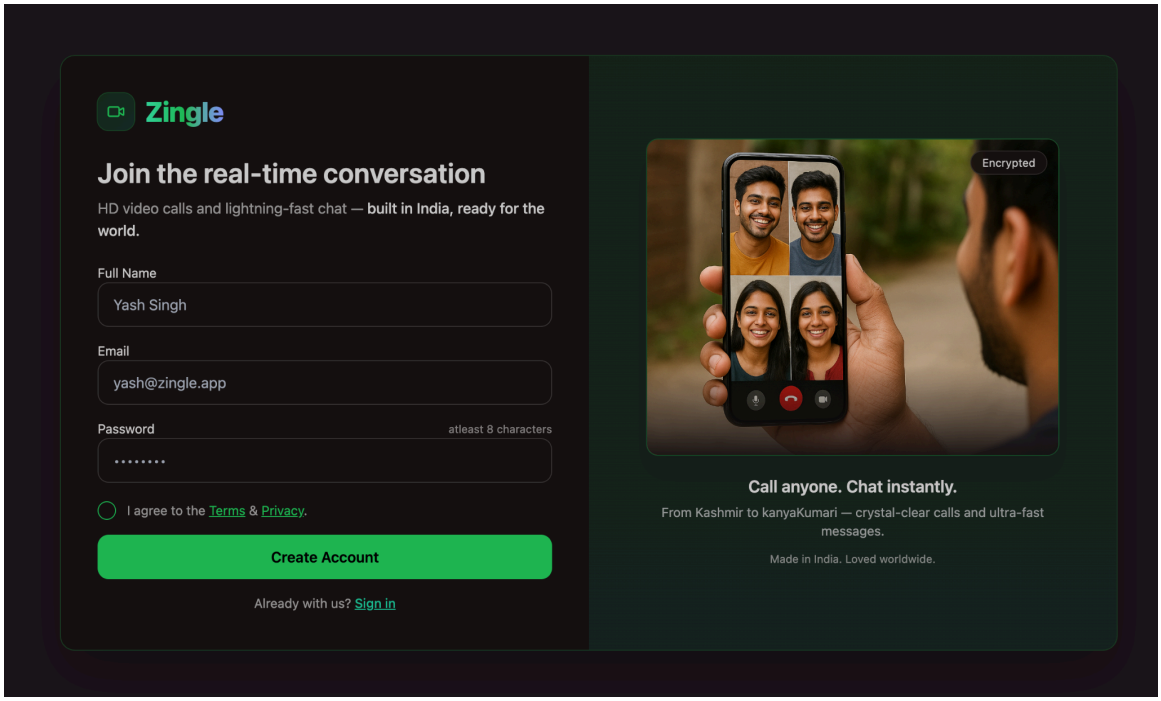
Real-time chat does not save every event manually → it is handled on Stream Chat Cloud.

4.2 Module Description

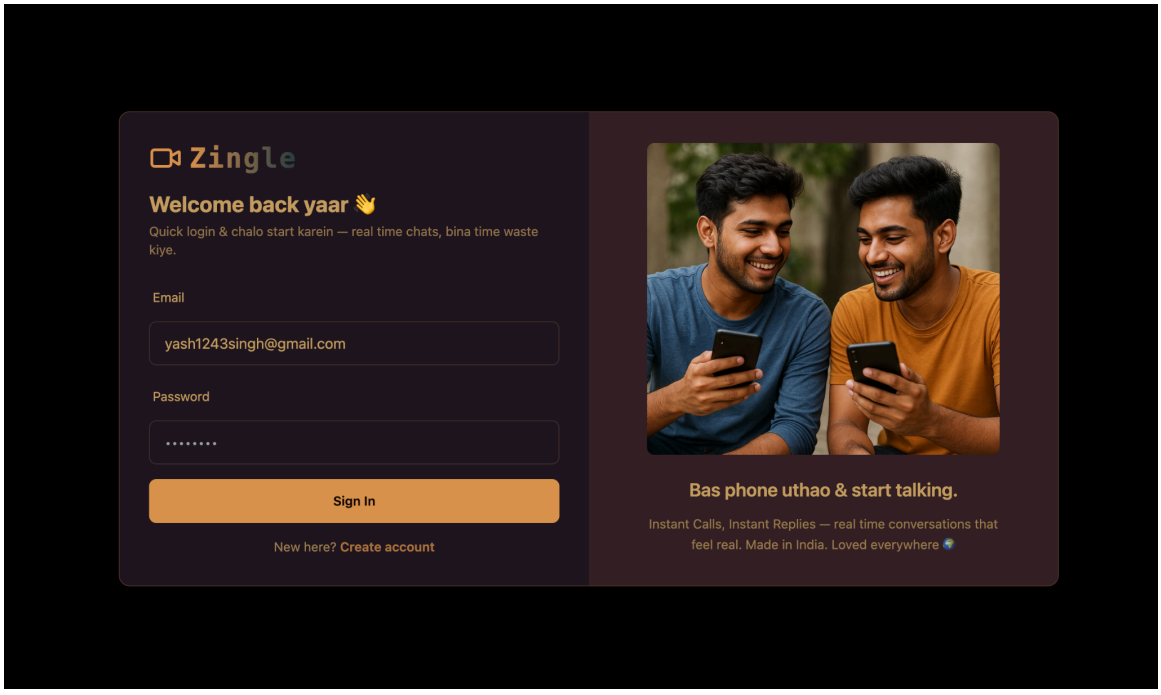
Module Name	Description
Authentication Module	Handles signup, login, logout using JWT & Cookies
Onboarding Module	User fills profile details like bio, languages, location
Friends Module	Send Request, Accept Request, List Friends
Chat Module	Two users can privately chat in one channel
Stream Token Module	Backend generates secured Stream Token for each logged-in user
Video Call Module	Generates dynamic room URL and starts WebRTC call

4.3 Flowcharts/ Snaps of Web pages

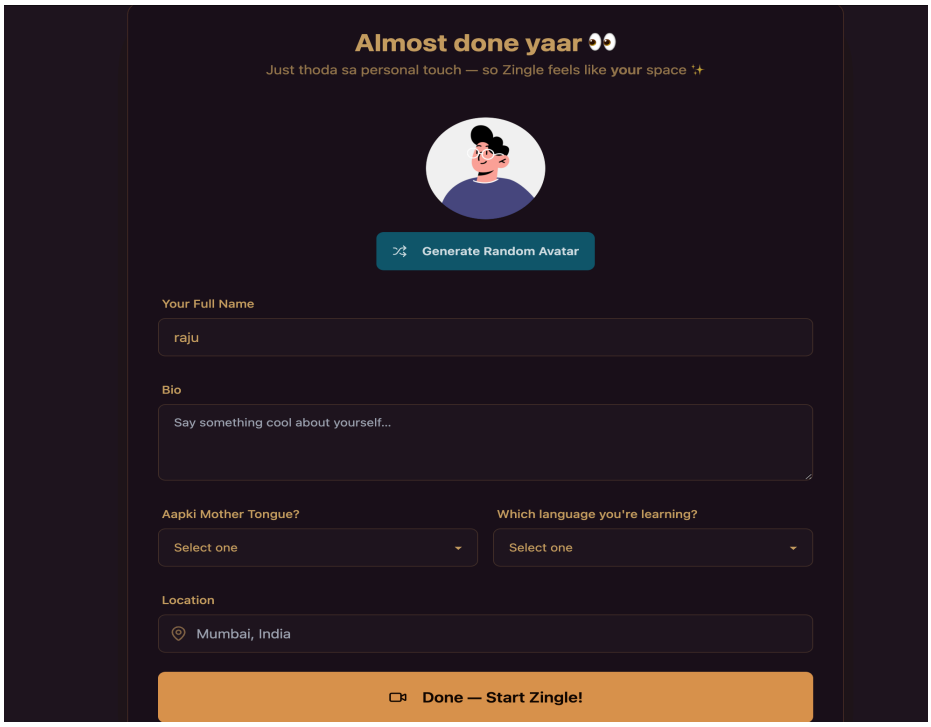
1.) SignUp Page



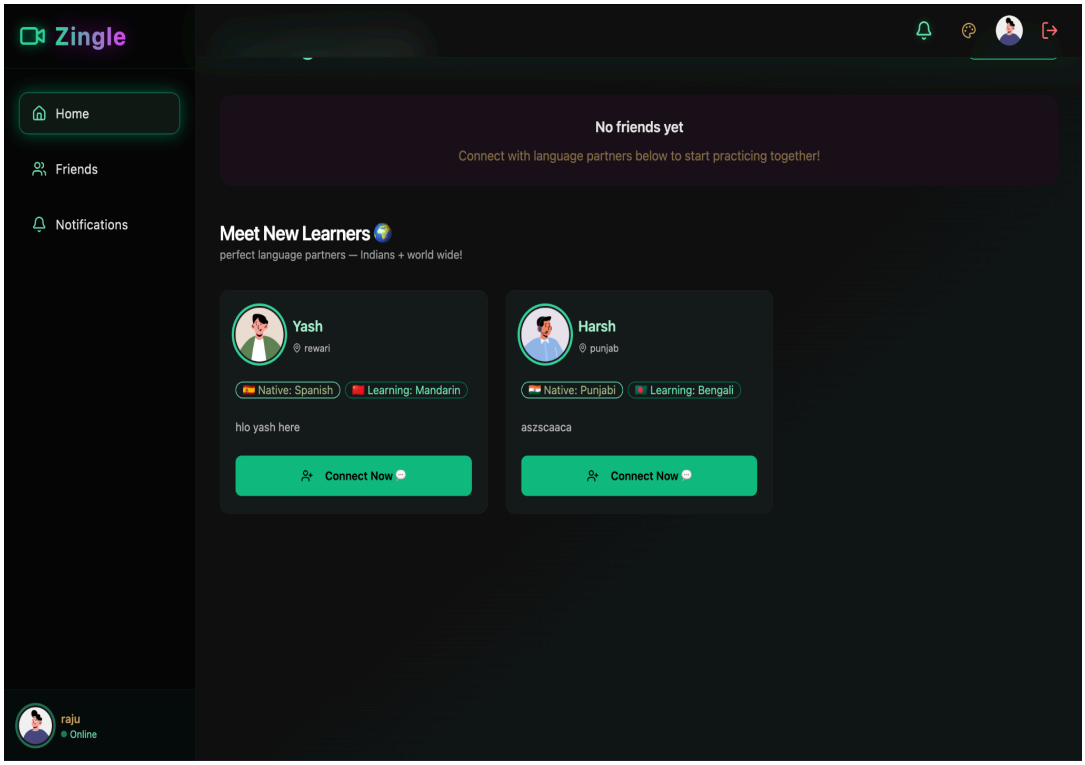
2.) Login Page



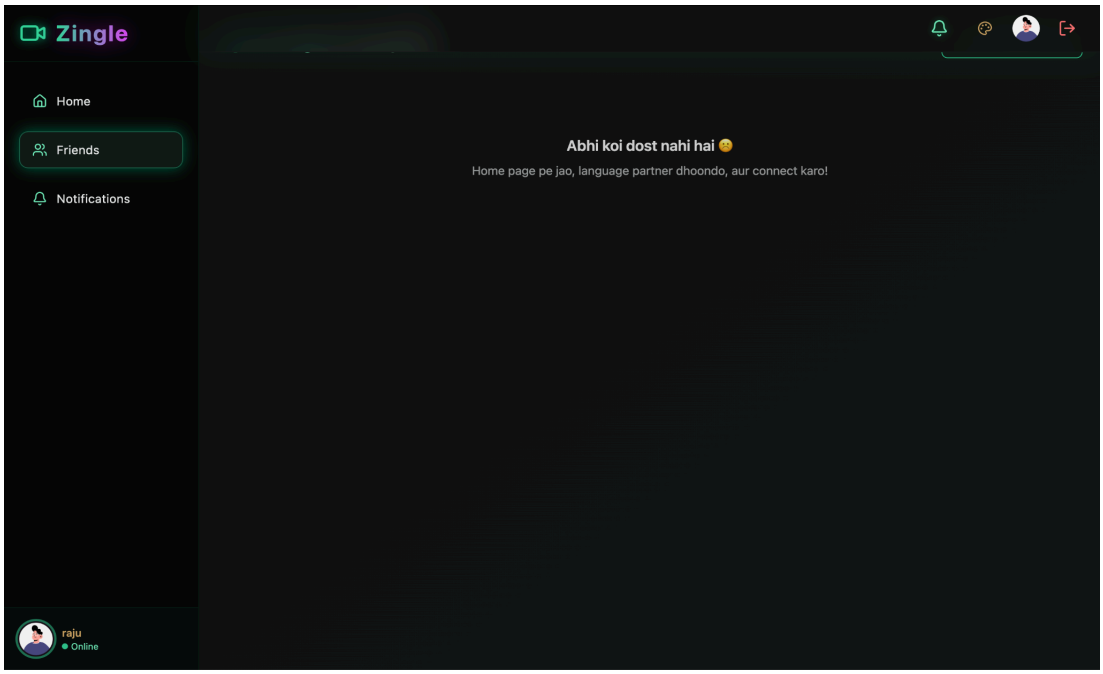
3.) Onboarding Page



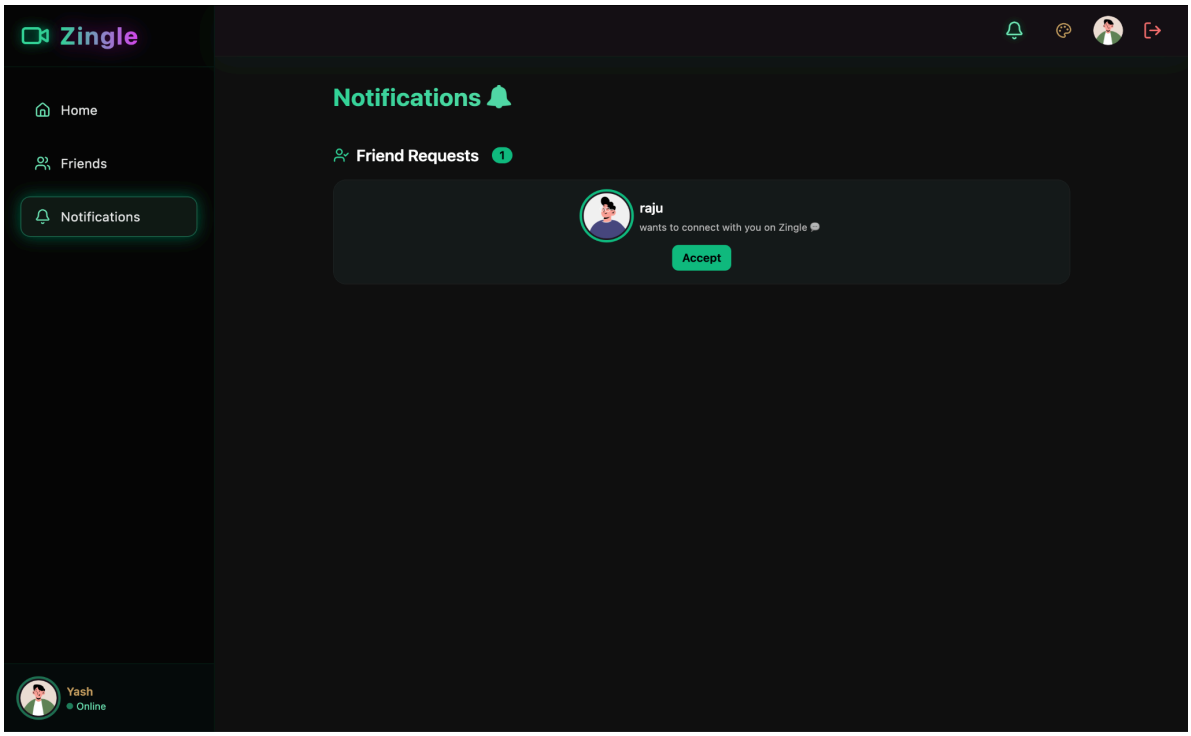
4.) Home Page



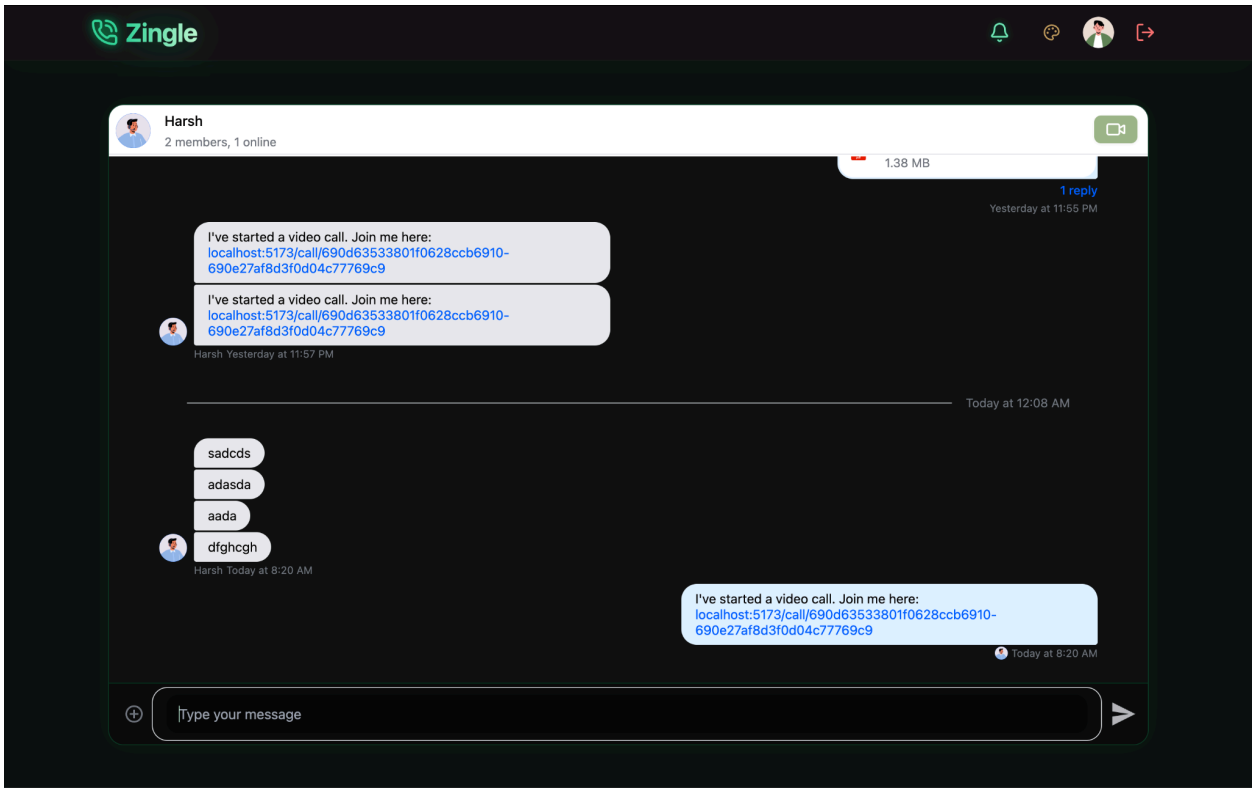
5.) Friends Page



6.) Notifications Page



7.) Chat Page



4.4 Development Process

Frontend Development

- React components approach, pages developed one-by-one
- Tailwind + DaisyUI used for instant modern UI
- React Router for page navigation

- TanStack Query for all API calls, caching, instant UI updates

- Stream Chat React SDK integrated to render modern messaging UI

Backend Development

- Express.js routes created module-wise
- JWT stored in HTTP-only cookies for safe session
- Middleware `protectRoute` used to verify user before accessing API endpoints
- Mongoose Schema created for Users & Friend Requests
- Stream API integrated inside backend to:
    - create Stream User on signup
    - generate Stream Token on chat page load

Chapter 5: Results and Discussion

This chapter presents the outcomes of the Zingle application, the environment used for testing, the testing approach, and performance evaluation.

5.1 Experimental Setup

Component	Configuration
Laptop Used	MacBook Air M1
Operating System	macOS
Frontend Dev Server	Vite
Backend Server	Node + Express
Database	MongoDB Atlas Cloud
Network Requirements	Stable Internet for Realtime events
Browser Tested On	Google Chrome

The entire system was tested using the local environment first and then later attempted for deployment on cloud.

5.2 Evaluation Metrics

Parameter	Target
UI Response Time	Must feel instant
Authentication Speed	< 1 second
Chat Delivery Time	Real-time less than 200ms delay
Video Call Setup Time	< 2 seconds to join call
Database Read/Write	Should be consistent under multiple requests

We do not measure “accuracy” (since it’s not ML project) but focus mainly on:

- **Latency** (Delay time in sending/receiving)
- **Reactivity** (Any change reflects instantly)
- **Scalability** (Stream can scale to millions of messages)
- **Reliability** (JWT ensures secure login sessions)

5.3 Results Analysis

Feature	Result Achieved
Signup/Login	Working using JWT
Profile Onboarding	Working with random avatar selection
Friend System	Send / Accept friend requests works perfectly
Chat System	Real-time streaming messages working
Video Call Link Sharing	User can generate dynamic call link
UI Design	Modern, clean, and professional interface

Overall Result Summary:

- Zingle successfully achieved its main objective of connecting people using enhanced chat and video calling features.
- The integration of **Stream API** gave industry-level real-time infrastructure without writing heavy WebRTC code manually.

Final Discussion:

Zingle behaves like a combination of **Instagram + WhatsApp concept** where users can:

- add friends
- chat in real-time
- switch to video call anytime

Performance is smooth due to:

- **Frontend caching** using TanStack Query
- **Cloud-based chat infra** by Stream
- **Optimized UI** with Tailwind & DaisyUI

Chapter 6: Conclusion and Future Work

6.1 Conclusion

Zingle – Video Calling, Realtime Chat & Social App – successfully demonstrates a modern communication platform with seamless interactions, secure authentication, and smooth real-time experience.

By integrating Stream APIs and using a scalable MERN-based architecture, this app achieves a professional level of performance similar to industrial social media platforms. The project also helped explore key modern development concepts such as cloud databases, JWT session management, real-time streaming, UI/UX improvements, and component-driven React architecture.

Overall, the project meets its objectives and provides an interactive & meaningful digital communication channel.

6.2 Limitations

Even though the application works well in real-time, there are some limitations:

Limitation	Description
Deployment complexity	Managing FE + BE + Stream secrets requires careful hosting setup
Basic call UI	Video calling is currently link based, not in-page embed call UI
Limited user discovery	Only friend suggestion system exists; no community / groups yet
Authentication only email-based	No social logins like Google/OAuth integrated

6.3 Future Scope

The project can be further extended to a full-fledged Social Media System by adding:

- Direct **in-browser video calling UI** with participant grid layout
- **Voice Calls** integration alongside video calls
- **Stories / Reels sharing** similar to Instagram
- **AI based friend suggestion** using ML (language match + interest match)
- **Push Notifications** (web & mobile)
- **Mobile App Version** using React Native
- **Dark / Light Dynamic Theming** with more personalization

If expanded systematically, Zingle can grow into a strong competitor for existing chat & calling apps.

And,Our **Zingle** is live at :
https://zingle-the-calling-and-chatting-app-3tla.onrender.com/

References

1. React Official Documentation – https://react.dev
2. Express.js Official Documentation – https://expressjs.com
3. MongoDB Documentation – https://www.mongodb.com/docs
4. Mongoose ORM Library Documentation – https://mongoosejs.com/docs
5. Stream Chat (Chat + Video Calling API) – Official Docs: https://getstream.io/
6. TanStack Query (React Query) Documentation – https://tanstack.com/query/latest
7. TailwindCSS Official Documentation – https://tailwindcss.com/docs
8. DaisyUI Components Documentation – https://daisyui.com
9. Node.js Documentation – https://nodejs.org/en/docs
10. JWT Authentication Documentation – https://jwt.io
11. Render Deployment Documentation – https://render.com/docs